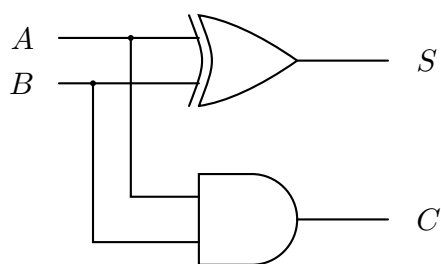


数字电路

(学 习 笔 记)



魏舟

2026 秋季学期

Stay hungry. Stay foolish.

求知若饥，虚心若愚

——史蒂夫·乔布斯 (Steve Jobs)

目录

第一章	数制与 BCD 码	2
1.1	常用数制	2
1.1.1	常用数制一览	2
1.1.2	二进制与十进制之间的转换	3
1.1.3	二进制算术运算	3
1.1.4	带符号数的表示：原码、反码和补码	4
1.2	几种简单的编码	6
1.2.1	二-十进制码 (BCD 码)	6
1.2.2	格雷码 (Gray 码)	8
1.2.3	计算机字符集	8
第二章	布尔代数	9
2.1	基本逻辑运算	9
2.1.1	与逻辑运算	9
2.1.2	或逻辑运算	10
2.1.3	非逻辑运算	11
2.2	复合逻辑运算	11
2.2.1	与非逻辑	11
2.2.2	或非逻辑	12
2.2.3	与或非逻辑	12
2.2.4	异或逻辑	14
2.2.5	同或逻辑	15
2.3	逻辑电平及正、负逻辑	16
2.4	基本定律和规则	16

2.4.1	逻辑函数的相等	16
2.4.2	基本定律	17
2.4.3	逻辑代数的三条规则	17
2.5	常用公式	18
2.5.1	消去律	18
2.5.2	吸收律 1	19
2.5.3	吸收律 2	19
2.5.4	包含律	19
2.5.5	关于异或和同或运算	20
2.6	逻辑函数的标准形式	21
2.6.1	函数的“与-或”式和“或-与”式	21
2.6.2	最小项	21
2.6.3	最大项	22
2.6.4	最小项和最大项的关系	23
2.6.5	逻辑函数的最小项之和形式	23
2.6.6	逻辑函数的最大项之积形式	23
2.6.7	两种标准形式之间的关系	24
2.7	真值表与逻辑表达式	24
2.7.1	由逻辑函数式列真值表	24
2.7.2	由真值表写逻辑函数式	25
2.8	逻辑函数的化简	25
2.8.1	公式法	25
2.8.2	卡诺图法	26
第三章	组合逻辑电路基础	30
3.1	由基本逻辑门构成的组合电路的设计与分析	30
3.1.1	一般分析方法	30
3.1.2	一般设计方法	31
3.2	MSI 构成的组合逻辑电路	33
3.2.1	自顶向下的模块化设计方法	33
3.2.2	编码器	35

前言

本笔记按课程内容整理，目前包括第一章「数制与 BCD 码」、第二章「布尔代数」，其余章节将陆续补入。

整理原则：概念与公式沿用课件原文表述，仅作版式重排；课件中被提取工具打散的公式已按标准写法还原。

第一章 数制与 BCD 码

数制（进位计数制）：用进位的方法来计数，包括**计数符号（数码）**和**进位规则**两个方面。常用的有十进制、二进制、八进制、十六进制等。

1.1 常用数制

1.1.1 常用数制一览

表 1.1: 常用数制一览

数制	计数符号	进位规则	按权展开式
十进制	0 ~ 9	逢十进一	$(N)_{10} = \sum_{i=-m}^{n-1} a_i \times 10^i$
二进制	0, 1	逢二进一	$(N)_2 = \sum_{i=-m}^{n-1} a_i \times 2^i$
八进制	0 ~ 7	逢八进一	$(N)_8 = \sum_{i=-m}^{n-1} a_i \times 8^i$
十六进制	0 ~ 9、A ~ F	逢十六进一	$(N)_{16} = \sum_{i=-m}^{n-1} a_i \times 16^i$

其中 a_i 为系数、 R^i 为权（ R 为基数）。

例：

$$1987.45 = 1 \times 10^3 + 9 \times 10^2 + 8 \times 10^1 + 7 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} \quad (1.1)$$

$$(6D.4B)_{16} = 6 \times 16^1 + 13 \times 16^0 + 4 \times 16^{-1} + 11 \times 16^{-2} \quad (D = 13, B = 11) \quad (1.2)$$

$$(63.45)_8 = 6 \times 8^1 + 3 \times 8^0 + 4 \times 8^{-1} + 5 \times 8^{-2} \quad (1.3)$$

数字电路中采用二进制的原因：

1. 数字装置简单可靠；
2. 二进制数运算规则简单；
3. 数字电路既可以进行算术运算，也可以进行逻辑运算。

1.1.2 二进制与十进制之间的转换

(1) 二进制数转换为十进制数（按权展开法）

$$\begin{aligned}
 (1011.101)_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
 &\quad + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\
 &= 8 + 2 + 1 + 0.5 + 0.125 = (11.625)_{10}
 \end{aligned} \tag{1.4}$$

(2) 十进制数转换为二进制数（提取 2 的幂法）

$$\begin{aligned}
 (45.5)_{10} &= 32 + 8 + 4 + 1 + 0.5 \\
 &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} \\
 &= (101101.1)_2
 \end{aligned} \tag{1.5}$$

数制转换还可以采用**基数连除、连乘**（除基数取余、乘基数取整）等方法。

1.1.3 二进制算术运算

算术运算规则与十进制相同，区别是**逢二进一**。其特点是：加、减、乘、除全部可以用**移位**和**相加**两种操作实现，电路结构简化，所以数字电路中普遍采用二进制算术运算。

加法：逐位相加并处理进位。

$$S_0 = A_0 + B_0 \text{ (进位 } C_1), \quad S_1 = A_1 + B_1 + C_1 \text{ (进位 } C_2), \quad \dots \tag{1.6}$$

设 $A = A_3A_2A_1A_0$ 、 $B = B_3B_2B_1B_0$ ，两数之和为 $S = S_3S_2S_1S_0$ ，则

$$A + B = C_4S_3S_2S_1S_0 \tag{1.7}$$

减法：将减数 B 变成补码后与被减数 A 相加，若有进位则舍去，其和即为两数之差。例：

$$\begin{aligned}
 Y &= (1000)_2 - (0100)_2 = (1000)_2 + [(1011)_2 + 1] \\
 &= (1000)_2 + (1100)_2 = (10100)_2 \xrightarrow{\text{舍去进位}} (0100)_2
 \end{aligned} \tag{1.8}$$

乘法：乘以 2 相当于**左移一位**。方法有：(1) 按十进制的乘法过程；(2) 采用移位相加的方法。

除法：除以 2^k 相当于**右移 k 位**。例 ($k = 2$, 即除以 4)：

$$(00001011)_2 \div (100)_2 = (00000010)_2 (\text{商}), \quad \text{余数} = (11)_2 \quad (1.9)$$

1.1.4 带符号数的表示：原码、反码和补码

计算机中的带符号数表示为**符号位 + 真值**（合称**机器数**），符号位用“0”表示正、“1”表示负。

定义 1.1 (原码) 真值 X 的原码记为 $[X]_{\text{原}}$ ：最高位为符号位，其余为真值部分；不论数的正负，数值部分均保持原真值不变。

优点：正负号与原真值之间的对应关系简单，容易理解；

缺点：用原码进行加减运算比较困难，**0 的表示不唯一**—— $+0 = 0\ 0000000$, $-0 = 1\ 0000000$ 。

定义 1.2 (反码) 真值 X 的反码记为 $[X]_{\text{反}}$ ：若 $X > 0$, 则 $[X]_{\text{反}} = [X]_{\text{原}}$ ；若 $X < 0$, 则符号位不变、数值部分按位取反。**0 的反码也不唯一**： $[+0]_{\text{反}} = 00000000$, $[-0]_{\text{反}} = 11111111$ 。

定义 1.3 (补码) 真值 X 的补码记为 $[X]_{\text{补}}$ ：若 $X > 0$, 则 $[X]_{\text{补}} = [X]_{\text{反}} = [X]_{\text{原}}$ ；若 $X < 0$, 则

$$[X]_{\text{补}} = [X]_{\text{反}} + 1 \quad (1.10)$$

补码的符号位与 $[X]_{\text{原}}$ 相同，**0 的表示唯一**，且可将减法运算转换为加法运算。

例 ($X = -52 = -0110100$):

$$[X]_{\text{原}} = 1\ 0110100, \quad [X]_{\text{反}} = 1\ 1001011, \quad [X]_{\text{补}} = [X]_{\text{反}} + 1 = 11001100 \quad (1.11)$$

0 的编码：8 位原码中 $+0 = 0\ 0000000$ 、 $-0 = 1\ 0000000$ ，不唯一；补码表示唯一：

$$[-0]_{\text{补}} = [-0]_{\text{反}} + 1 = 11111111 + 1 = 1\ 00000000 \xrightarrow{8\text{ 位字长舍去进位}} 00000000 \quad (1.12)$$

特殊数 10000000：

- 原码中定义为 -0 ；
- 反码中定义为 -127 ；
- 补码中定义为 -128 ；
- 作为无符号数时 $(10000000)_2 = 128$ 。

表示范围 (8 位)：

表 1.2: 8 位数的表示范围

编码	表示范围
原码 / 反码	$-127 \sim +127$
补码	$-128 \sim +127$
无符号数	$0 \leq X \leq 2^n - 1$

补码运算：通过引进补码，可将减法运算转换为加法运算，即

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}, \quad [X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} \quad (1.13)$$

例 1.1. 补码加法 $X = -0110100$, $Y = +1110100$, 求 $X + Y$ 。

解.

$$[X]_{\text{补}} = 11001100, \quad [Y]_{\text{补}} = 01110100 \quad (1.14)$$

所以

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}} = 11001100 + 01110100 = 01000000 \quad (1.15)$$

符号位讨论——用二进制补码求 13 ± 10 、 -13 ± 10 (6 位字长):

$$\begin{aligned}13 + 10 &= 001101 + 001010 = 010111 = 23, \\13 - 10 &= 001101 + 110110 = 000011 = 3, \\-13 + 10 &= 110011 + 001010 = 111101 = -3, \\-13 - 10 &= 110011 + 110110 = 101001 = -23\end{aligned}\tag{1.16}$$

无符号数的溢出：无符号数的范围为 $0 \leq X \leq 2^n - 1$ ，若运算结果超出这个范围，则产生**溢出**。溢出判断准则：运算时，当**最高位向更高位有进位（或借位）**时则产生溢出。

1.2 几种简单的编码

1.2.1 二-十进制码 (BCD 码)

定义 1.4 (二-十进制码 (BCD 码)) 用四位二进制代码来表示一位十进制数码，这样的代码称为二-十进制码 (BCD 码, Binary Coded Decimal codes)。四位二进制有 16 种不同组合，可以在这 16 种代码中任选 10 种表示十进制数的 10 个不同符号，选择方法不同，就得到不同的编码形式。常见的有 8421 码、5421 码、2421 码、余 3 码等。

表 1.3: 常用 BCD 码

十进制数	8421 码	5421 码	2421 码	余 3 码
0	0000	0000	0000	0011
1	0001	0001	0001	0100
2	0010	0010	0010	0101
3	0011	0011	0011	0110
4	0100	0100	0100	0111
5	0101	1000	1011	1000
6	0110	1001	1100	1001
7	0111	1010	1101	1010
8	1000	1011	1110	1011
9	1001	1100	1111	1100

定义 1.5 (有权 BCD 码) 每位数码都有确定的位权的码，例如 8421 码、5421 码、2421 码。

例: 5421 码 1011 代表 $5+0+2+1 = 8$; 2421 码 1100 代表 $2+4+0+0 = 6$ 。

5421 码与 2421 码的编码不唯一，例如 2421 码 0110 也可表示 6。

由表 1.3 可见：

1. 8421 码与代表 0 ~ 9 的二进制数一一对应；
2. 5421 码前 5 个码与 8421 码相同，后 5 个码在前 5 个码的基础上加 1000 构成；
3. 2421 码前 5 个码与 8421 码相同，后 5 个码以中心对称取反，这样的码称为**自反代码**。例如 $4 \rightarrow 0100$ 、 $5 \rightarrow 1011$ ； $0 \rightarrow 0000$ 、 $9 \rightarrow 1111$ 。

定义 1.6 (无权 BCD 码 (余 3 码)) 每位数码无确定的位权，例如余 3 码。余 3 码的编码规律为：在 8421 码上加 0011。例如 6 的余 3 码为 $0110 + 0011 = 1001$ 。

1.2.2 格雷码 (Gray 码)

格雷码为无权码，特点为：相邻两个代码之间仅有一位不同，其余各位均相同，具有这种特点的代码称为**循环码**，格雷码是循环码。

格雷码与四位二进制码之间的关系：设四位二进制码为 $B_3B_2B_1B_0$ ，格雷码为 $R_3R_2R_1R_0$ ，则

$$R_3 = B_3, \quad R_2 = B_3 \oplus B_2, \quad R_1 = B_2 \oplus B_1, \quad R_0 = B_1 \oplus B_0 \quad (1.17)$$

其中 $\oplus$ 为异或运算符，运算规则：两运算数相同结果为“0”，不同结果为“1”。

1.2.3 计算机字符集

各种文字和符号的总称，包括各国家文字、标点符号、图形符号、数字等。常见字符集：ASCII、GB2312、BIG5、GB18030、Unicode。

第二章 布尔代数

数字电路的基础为逻辑代数，由英国数学家 George Boole 在 1847 年提出，逻辑代数也称布尔代数。

2.1 基本逻辑运算

在逻辑代数中，变量常用字母 $A, B, C, \dots, Y, Z$ 、 $a, b, c, \dots, x, y, z$ 等表示，变量的取值只能是“0”或“1”。

逻辑代数中只有三种基本逻辑运算，即“与”、“或”、“非”。

2.1.1 与逻辑运算

定义 2.1（与逻辑运算） 只有决定一事件的全部条件都具备时，这件事才成立；如果有一个或一个以上条件不具备，则这件事就不成立。这样的因果关系称为“与”逻辑关系，其函数式为 $F = A \cdot B = AB$ 。

与门逻辑功能概括：有“0”出“0”，全“1”出“1”。

与门逻辑符号如下：

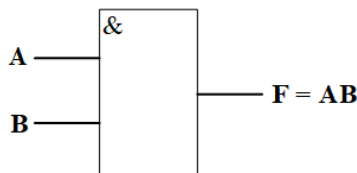


图 2.1: 与门逻辑符号

与逻辑真值表如下：

表 2.1: 与逻辑真值表

A	B	$F = AB$
0	0	0
0	1	0
1	0	0
1	1	1

2.1.2 或逻辑运算

定义 2.2 (或逻辑运算) 在决定一事件的各种条件中，只要有一个或一个以上条件具备时，这件事就成立；**只有所有的条件都不具备时**，这件事才**不成立**。这样的因果关系称为“或”逻辑关系，其函数式为 $F = A + B$ 。

或门逻辑功能概括：**有“1”出“1”，全“0”出“0”**。

或门逻辑符号如下：

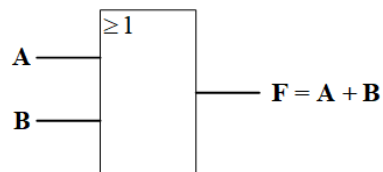


图 2.2: 或门逻辑符号

或逻辑真值表如下：

表 2.2: 或逻辑真值表

A	B	$F = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

注：与门和或门均可以有多个输入端。

2.1.3 非逻辑运算

定义 2.3 (非逻辑运算) 假定事件 F 成立与否同条件 A 的具备与否有关：若 A 具备，则 F 不成立；若 A 不具备，则 F 成立。 F 和 A 之间的这种因果关系称为“非”逻辑关系，其函数式为 $F = \overline{A}$ 。

非门逻辑符号如下：

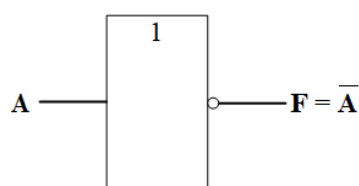


图 2.3: 非门逻辑符号

非逻辑真值表如下：

表 2.3: 非逻辑真值表

A	$F = \overline{A}$
0	1
1	0

2.2 复合逻辑运算

2.2.1 与非逻辑

与非逻辑由与逻辑和非逻辑组合而成，函数式为 $F = \overline{A \cdot B} = \overline{AB}$ 。

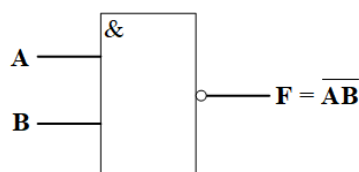


图 2.4: 与非门逻辑符号

与非逻辑真值表如下：

表 2.4: 与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

2.2.2 或非逻辑

或非逻辑由或逻辑和非逻辑组合而成，函数式为 $F = \overline{A + B}$ 。

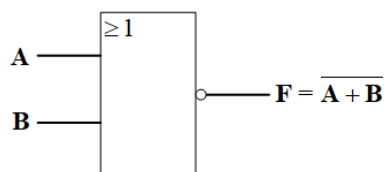


图 2.5: 或非门逻辑符号

或非逻辑真值表如下：

表 2.5: 或非逻辑真值表

A	B	$F = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

2.2.3 与或非逻辑

与或非逻辑由与、或、非三种逻辑组合而成，函数式为 $F = \overline{AB + CD}$ 。

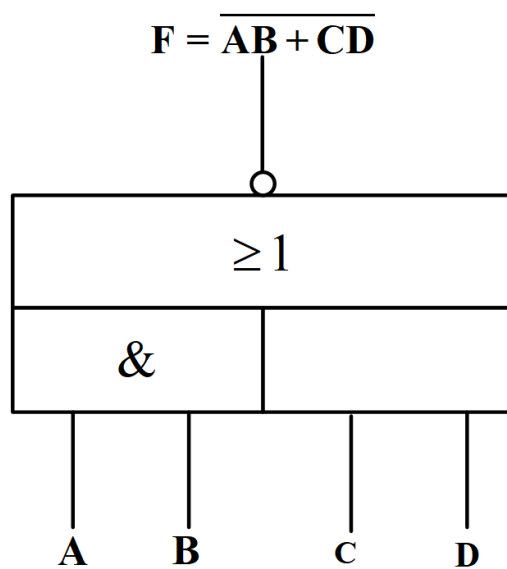


图 2.6: 与或非门逻辑符号

与或非逻辑真值表如下：

表 2.6: 与或非逻辑真值表

A	B	C	D	$F = \overline{AB + CD}$
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

2.2.4 异或逻辑

异或逻辑的函数式为 $F = \overline{A}B + A\overline{B} = A \oplus B$ ，口诀：相同得“0”，相异得“1”。

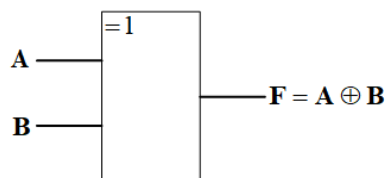


图 2.7: 异或门逻辑符号

异或逻辑真值表如下：

表 2.7: 异或逻辑真值表

A	B	$F = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

2.2.5 同或逻辑

同或逻辑的函数式为 $F = \overline{A}\overline{B} + AB = A \odot B$ ，口诀：相同得“1”，相异得“0”。

同或和异或互为反函数，即

$$A \oplus B = \overline{A \odot B} \quad (2.1)$$

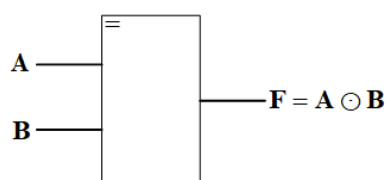


图 2.8: 同或门逻辑符号

同或逻辑真值表如下：

表 2.8: 同或逻辑真值表

A	B	$F = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

2.3 逻辑电平及正、负逻辑

门电路的输入、输出为二值信号，用“0”和“1”表示，这里的“0”、“1”一般用两个不同电平值来表示。

定义 2.4 (正逻辑与负逻辑)

- **正逻辑**：若用高电平 V_H 表示逻辑“1”，用低电平 V_L 表示逻辑“0”，则称为正逻辑约定，简称正逻辑；
- **负逻辑**：若用高电平 V_H 表示逻辑“0”，用低电平 V_L 表示逻辑“1”，则称为负逻辑约定，简称负逻辑。

一般采用正逻辑表示。

注： 对一个特定的逻辑门，采用不同的逻辑表示时，其门的名称也就不同。

2.4 基本定律和规则

2.4.1 逻辑函数的相等

设有两个逻辑函数

$$F_1 = f_1(A_1, A_2, \dots, A_n), \quad F_2 = f_2(A_1, A_2, \dots, A_n) \quad (2.2)$$

如果对于 $A_1, A_2, \dots, A_n$ 的任何一组取值（共 2^n 组）， F_1 和 F_2 均相等，则称 F_1 和 F_2 相等。

因此，如两个函数的真值表相等，则这两个函数一定相等。

2.4.2 基本定律

表 2.9: 逻辑代数的基本定律

定律	与形式	或形式
① 0-1 律	$A \cdot 0 = 0$	$A + 1 = 1$
② 自等律	$A \cdot 1 = A$	$A + 0 = A$
③ 重叠律	$A \cdot A = A$	$A + A = A$
④ 互补律	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
⑤ 交换律	$A \cdot B = B \cdot A$	$A + B = B + A$
⑥ 结合律	$A(BC) = (AB)C$	$A + (B + C) = (A + B) + C$
⑦ 分配律	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
⑧ 反演律	$\overline{A + B} = \bar{A} \cdot \bar{B}$	$\overline{AB} = \bar{A} + \bar{B}$

反演律也称**德·摩根定理**，是一个非常有用的定理。

2.4.3 逻辑代数的三条规则

代入规则

任何一个含有变量 x 的等式，如果将所有出现 x 的位置，都用一个逻辑函数式 F 代替，则等式仍然成立。

反演规则

设 F 为任意逻辑表达式，若将 F 中所有运算符、常量及变量作如下变换：

·	+	0	1	原变量	反变量
↓	↓	↓	↓	↓	↓
+	·	1	0	反变量	原变量

图 2.9: 反函数的变换规则

则所得新的逻辑式即为 F 的反函数，记为 $\bar{F}$ 。

注意：

1. 保持原式运算的优先次序；
2. 原式中的不属于单变量上的非号不变。

对偶规则

设 F 为任意逻辑表达式，若将 F 中所有运算符和常量作如下变换：

·	+	0	1
↓	↓	↓	↓
+	·	1	0

图 2.10: 对偶式的变换规则

则所得新的逻辑表达式即为 F 的对偶式，记为 F' 。

2.5 常用公式

2.5.1 消去律

定理 2.5 (消去律)

$$AB + A\overline{B} = A \quad (2.3)$$

证明：

$$AB + A\overline{B} = A(B + \overline{B}) = A \cdot 1 = A \xrightarrow{\text{对偶关系}} (A + B)(A + \overline{B}) = A \quad (2.4)$$

□

2.5.2 吸收律 1

定理 2.6 (吸收律 1)

$$A + AB = A \quad (2.5)$$

证明:

$$A + AB = A(1 + B) = A \cdot 1 = A \xrightarrow{\text{对偶关系}} A(A + B) = A \quad (2.6)$$

□

2.5.3 吸收律 2

定理 2.7 (吸收律 2)

$$A + \overline{A}B = A + B \quad (2.7)$$

证明:

$$\begin{aligned} A + \overline{A}B &= (A + \overline{A})(A + B) = 1 \cdot (A + B) \xrightarrow{\text{对偶关系}} A(\overline{A} + B) = AB \\ &= A + B \end{aligned} \quad (2.8)$$

□

2.5.4 包含律

定理 2.8 (包含律)

$$AB + \overline{A}C + BC = AB + \overline{A}C \quad (2.9)$$

证明:

$$\begin{aligned} &AB + \overline{A}C + BC \\ &= AB + \overline{A}C + (A + \overline{A})BC \\ &= AB + \overline{A}C + ABC + \overline{A}BC \\ &= AB(1 + C) + \overline{A}C(1 + B) \end{aligned} \quad (2.10)$$

由对偶关系，可得

$$(A + B)(\overline{A} + C)(B + C) = (A + B)(\overline{A} + C) \quad (2.11)$$

□

2.5.5 关于异或和同或运算

性质 2.9 (对偶数个变量)

$$A_1 \oplus A_2 \oplus \cdots \oplus A_n = \overline{A_1 \odot A_2 \odot \cdots \odot A_n} \quad (2.12)$$

性质 2.10 (对奇数个变量)

$$A_1 \oplus A_2 \oplus \cdots \oplus A_n = A_1 \odot A_2 \odot \cdots \odot A_n \quad (2.13)$$

异或和同或的其他性质如表 2.10 所示。

表 2.10: 异或和同或的其他性质

异或	同或
$A \oplus 0 = A$	$A \odot 1 = A$
$A \oplus 1 = \overline{A}$	$A \odot 0 = \overline{A}$
$A \oplus A = 0$	$A \odot A = 1$
$A \oplus (B \oplus C) = (A \oplus B) \oplus C$	$A \odot (B \odot C) = (A \odot B) \odot C$
$A(B \oplus C) = AB \oplus AC$	$A + (B \odot C) = (A + B) \odot (A + C)$

2.6 逻辑函数的标准形式

2.6.1 函数的“与-或”式和“或-与”式

定义 2.11 (“与-或”式) “与-或”式指一个函数表达式中包含若干个“与”项，这些“与”项的“或”表示这个函数，如

$$F(A, B, C, D) = A + B\bar{C} + ABC\bar{D} \quad (2.14)$$

定义 2.12 (“或-与”式) “或-与”式指一个函数表达式中包含若干个“或”项，这些“或”项的“与”表示这个函数，如

$$F(A, B, C, D) = (A + C + D)(\bar{B} + D)(A + \bar{B} + \bar{D}) \quad (2.15)$$

2.6.2 最小项

定义 2.13 (最小项) 最小项是“与”项，具有以下特点：

1. n 个变量构成的每个最小项一定包含 n 个因子的乘积项；
2. 每个变量必须以原变量或反变量形式作为因子出现一次且仅一次。

例如：两变量有 $2^2 = 4$ 个最小项： $A\bar{B}$ 、 $\bar{A}B$ 、 $\bar{A}\bar{B}$ 、 AB ；三变量有 $2^3 = 8$ 个最小项。

定义 2.14 (最小项的编号) 任一最小项用 m_i 表示， m 表示最小项，下标 i 为使该最小项为 1 的变量取值所对应的等效十进制数。

例如：使最小项 $A\bar{B}C$ 为 1 的取值 $A, B, C = 1, 0, 1$ ，二进制数 101 等效十进制数为 5，所以 $A\bar{B}C = m_5$ 。

性质 2.15 (最小项的性质)

1. 变量任取一组值，仅有一个最小项为 1，其他最小项为零；
2. n 变量的全体最小项之和为 1；
3. 不同的最小项相与，结果为 0；
4. 相邻最小项相“或”可以合并成一项，并消去一个变量因子。

定义 2.16 (最小项相邻) 两最小项如仅有一个变量因子不同，其他变量均相同，则称这两个最小项相邻。例如 $\overline{A}BC + ABC = BC$ 。任一 n 变量的最小项，必定和其他 n 个不同最小项相邻。

2.6.3 最大项

定义 2.17 (最大项) 最大项是“或”项，具有以下特点：

1. 每个最大项一定包含 n 个因子的“或”项；
2. 每个变量必须以原变量或反变量形式出现一次且仅一次。

例如：两变量有 $A + B$ 、 $A + \overline{B}$ 、 $\overline{A} + B$ 、 $\overline{A} \overline{B}$ 四个最大项。

定义 2.18 (最大项的编号) 任一最大项用 M_i 表示，下标 i 为使该最大项为 0 的变量取值所对应的等效十进制数。

例如：使最大项 $\overline{A} + B + C$ 为 0 的取值为 $A, B, C = 1, 0, 0$ ，二进制数 100 等效十进制数为 4，所以 $\overline{A} + B + C = M_4$ 。

性质 2.19 (最大项的性质)

1. 变量任取一组值，仅有一个最大项为 0，其他最大项为 1；
2. n 变量的全体最大项之积为 0；
3. 不同的最大项相或，结果为 1；

4. 两相邻的最大项相“与”可以合并成一项，并消去一个变量因子。
例如 $(A + B + \overline{C})(A + B + C) = A + B$ 。任一 n 变量的最大项，必定和其他 n 个不同最大项相邻。

2.6.4 最小项和最大项的关系

定理 2.20 (最小项与最大项的关系) 编号下标相同的最小项和最大项互为反函数，即

$$M_i = \overline{m_i} \quad (2.16)$$

2.6.5 逻辑函数的最小项之和形式

定理 2.21 (最小项之和形式) 最小项之和式为“与或”式，任一逻辑函数都可以表达为最小项之和的形式，而且是**唯一**的。例如

$$F(A, B, C) = \overline{A}B\overline{C} + A\overline{B}\overline{C} + ABC = \Sigma m(2, 4, 6) \quad (2.17)$$

例 2.1. 将 $F(A, B, C) = AB + \overline{A}C$ 化为最小项之和的形式：

$$\begin{aligned} F(A, B, C) &= AB + \overline{A}C \\ &= AB(C + \overline{C}) + \overline{A}C(B + \overline{B}) \\ &= ABC + AB\overline{C} + \overline{A}BC + \overline{A}\overline{B}C \\ &= \Sigma m(1, 3, 6, 7) \end{aligned} \quad (2.18)$$

2.6.6 逻辑函数的最大项之积形式

定理 2.22 (最大项之积形式) 最大项之积式为“或与”式，任一逻辑函数都可以表达为最大项之积的形式，而且是**唯一**的。例如

$$F(A, B, C) = (A + B + C)(A + \overline{B} + C)(\overline{A} + B + C) = \Pi M(0, 2, 4) \quad (2.19)$$

例 2.2. 将 $F(A, B, C) = (\overline{A} + C)(B + \overline{C})$ 化为最大项之积的形式：

$$\begin{aligned}
 F(A, B, C) &= (\overline{A} + C)(B + \overline{C}) \\
 &= (\overline{A} + C + B\overline{B})(B + \overline{C} + A\overline{A}) \\
 &= (\overline{A} + B + C)(\overline{A} + \overline{B} + C)(A + B + \overline{C})(\overline{A} + B + \overline{C}) \\
 &= \Pi M(1, 4, 5, 6)
 \end{aligned} \tag{2.20}$$

2.6.7 两种标准形式之间的关系

定理 2.23 (两种标准形式之间的关系) 若 $F = \Sigma m_i$, 则

$$\overline{F} = \sum_{j \neq i} m_j = \prod_{j \neq i} M_j \tag{2.21}$$

例 2.3. $F(A, B, C) = \Sigma(1, 3, 4, 6, 7) = \Pi(0, 2, 5)$

2.7 真值表与逻辑表达式

真值表与逻辑表达式都是表示逻辑函数的方法。

2.7.1 由逻辑函数式列真值表

例 2.4. 试列出逻辑函数式 $F(A, B, C) = AB + BC$ 的真值表。

方法一 (直接代入法)： 把 A, B, C 三个变量所有取值的组合分别代入函数式，算出函数值，填进真值表，适用于变量较少的情况。

方法二： 先将函数式 F 表示为最小项之和的形式：

$$\begin{aligned}
 F(A, B, C) &= AB + BC \\
 &= AB(C + \overline{C}) + BC(A + \overline{A}) \\
 &= ABC + AB\overline{C} + \overline{A}BC \\
 &= \Sigma m(3, 6, 7)
 \end{aligned} \tag{2.22}$$

最后根据最小项的性质，在真值表中对应于 ABC 取值为 011、110、111 处填“1”，其它位置填“0”，即

表 2.11: $F(A, B, C) = AB + BC$ 的真值表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

方法三：根据函数式 F 的含义，直接填表。

2.7.2 由真值表写逻辑函数式

方法 根据最小项的性质，用观察法，可直接从真值表写出函数的最小项之和表达式。

2.8 逻辑函数的化简

最简与或表达式标准

1. 所得与或表达式中，乘积项（与项）数目最少；
2. 每个乘积项中所含的变量数最少。

2.8.1 公式法

公式法化简就是反复运用逻辑代数公式消去多余的乘积项和每个乘积项中多余的因子，使函数式符合最简标准。常用方法如表 2.12 所示。

表 2.12: 公式法化简的常用方法

方法	利用公式	例
并项法	$AB + A\bar{B} = A$	$F = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + ABC = (\bar{A}\bar{B} + \bar{A}B)C + (A\bar{B} + AB)C = \bar{A}C + AC = C$
吸收法	$A + AB = A$	$F = A\bar{B} + A\bar{B}C = A\bar{B}$
消项法	$AB + \bar{A}C + BC = AB + \bar{A}C$	$F = AB + \bar{A}C + BC = AB + \bar{A}C$
消因子法	$A + \bar{A}B = A + B$	$F = A + \bar{A}BC = A + BC$
配项法	$A + \bar{A} = 1, A \cdot 1 = A$	$F = AB + \bar{A}C + BC = AB + \bar{A}C + (A + \bar{A})BC = AB(1 + C) + \bar{A}C(1 + B) = AB + \bar{A}C$

注：对比较复杂的函数式，要求熟练掌握上述方法，才能把函数化成最简。

2.8.2 卡诺图法

将逻辑函数用**卡诺图**表示，然后在卡诺图上进行化简。卡诺图是一种包含小方块的几何图形，每个小方块称为一个**单元**，每个单元对应一个最小项，两个相邻的最小项在卡诺图中也必须相邻。

卡诺图的构成

- 相邻的含义：① **几何相邻性**（左右紧挨或者上下相接）；② **对称相邻性**（图形中对称位置的单元是相邻的）；
- 行列标号按**循环码**（格雷码）排列。

三变量卡诺图与四变量卡诺图分别如图 2.11 和图 2.12 所示。

$A \setminus BC$	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

图 2.11: 三变量卡诺图

$AB \backslash CD$	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

图 2.12: 四变量卡诺图

二变量、五变量卡诺图类似，标号同样按循环码排列。

逻辑函数的卡诺图表示法

把各组变量值所对应的逻辑函数 F 的值填在对应的小方格中（卡诺图实质是真值表的另一种画法）。

在卡诺图上合并最小项的规则

- 任何两个标 1 方格相邻，可合并为 1 项，消去 1 个变量；
- 任何四个标 1 方格相邻，可合并为一项，消去 2 个变量（四个标 1 方格相邻的特点：①同在一行或一列；②同在一个田字格中）；
- 任何八个标 1 方格相邻，可并成一项，消去 3 个变量。

一般规律：在 n 个变量的卡诺图中，只有 2^i 个相邻的标 1 方格（必须排列成方形格或矩形格）才能圈在一起合并为一项，该项保留原来各项中 $n - i$ 个相同的变量，消去 i 个不同的变量。

用卡诺图化简逻辑函数

最简标准：项数最少（圈数最少）；每项中变量数最少（圈尽可能大）。

例 2.5. 将 $F(A, B, C) = \Sigma m(3, 4, 5, 6, 7)$ 化为最简与或式。

$A \backslash BC$	00	01	11	10
0	0	0	1	0
1	1	1	1	1

图 2.13: 例 2.5 的卡诺图及圈法

解. 将 $A = 1$ 的 4 个标 1 方格圈为一组 (图中蓝色圈), 合并为乘积项 A ; 将 $BC = 11$ 的 m_3 、 m_7 两个标 1 方格圈为一组 (图中红色圈), 合并为乘积项 BC 。两圈已覆盖全部标 1 方格, 故最简与或式为

$$F(A, B, C) = A + BC \quad (2.23)$$

例 2.6. 将 $F(A, B, C, D) = \Sigma m(0, 1, 3, 7, 8, 10, 13)$ 化为最简与或式。

$AB \backslash CD$	00	01	11	10
00	1	1	1	0
01	0	0	1	0
11	0	1	0	0
10	1	0	0	1

图 2.14: 例 2.6 的卡诺图及圈法

解. 先圈出孤立的标 1 方格, 再圈出覆盖其余标 1 方格的最大圈:

1. m_0, m_1 相邻, 圈为一组 (蓝色圈), 合并为 $\overline{A}\overline{B}\overline{C}$;
2. m_3, m_7 相邻, 圈为一组 (红色圈), 合并为 $\overline{A}CD$;
3. m_8, m_{10} 位于同一行两端、左右相邻 (卷绕相邻), 圈为一组 (橙色圈), 合并为 $\overline{A}\overline{B}\overline{D}$;
4. m_{13} 孤立, 单独圈出 (绿色圈), 为 $AB\overline{C}D$ 。

将各乘积项相加, 得最简与或式

$$F(A, B, C, D) = \overline{A}\overline{B}\overline{C} + \overline{A}CD + \overline{A}\overline{B}\overline{D} + AB\overline{C}D \quad (2.24)$$

化简中注意的问题

1. 每一个标 1 的方格必须至少被圈一次;
2. 每个圈中包含的相邻小方格数必须为 2 的整数次幂;
3. 为了得到尽可能大的圈, 圈与圈之间可以重叠;
4. 若某个圈中的所有标 1 方格已经完全被其他圈所覆盖, 则该圈为多余的。

用卡诺图求反函数的最简与或式

在卡诺图中合并标 0 方格，可得到反函数的最简与或式；常利用该方法求逻辑函数 F 的最简与或非式（将 $\bar{F}$ 上的非号移到右边）。

复杂逻辑函数

可将函数分解成多个部分，先将每个部分分别填入各自的卡诺图，然后通过卡诺图对应方格的运算（与、或、异或等）求出函数的卡诺图，再进行化简。

不完全确定的逻辑函数及其化简（无关项）

定义 2.24（无关项） 在某些实际数字电路中，逻辑函数的输出只和一部分最小项有确定对应关系，而和余下的最小项无关；余下的最小项无论写入函数式与否都不影响电路的逻辑功能。这些最小项称为**无关项**，用字母 d (don't care) 表示，对应的函数值记为 “ $\times$ ”。

包含无关项的逻辑函数称为**不完全确定的逻辑函数**。利用无关项往往可以把函数化得更简单。

例 2.7. 设计一个奇偶判别电路，输入为 8421BCD 码，输入为偶数时输出 0，输入为奇数时输出 1。由于 8421BCD 码中无 1010 ~ 1111 这 6 个码，电路禁止输入这 6 个码，它们对应的最小项为无关项：

$$F(A, B, C, D) = \Sigma m(1, 3, 5, 7, 9) + \Sigma d(10 \sim 15) \quad (2.25)$$

- 若不利用无关项（将卡诺图中的 $\times$ 均作 0 处理），化简结果为

$$F(A, B, C, D) = \bar{A}D + \bar{B}\bar{C}D$$

- 若利用无关项（将 $\times$ 按化简需要处理，有的作 0、有的作 1），化简结果为 $F(A, B, C, D) = D$ 。

第三章 组合逻辑电路基础

定义 3.1 (组合逻辑电路) 电路在任意时刻的输出状态仅由当前输入状态决定，与之前的输出状态和输入状态无关。

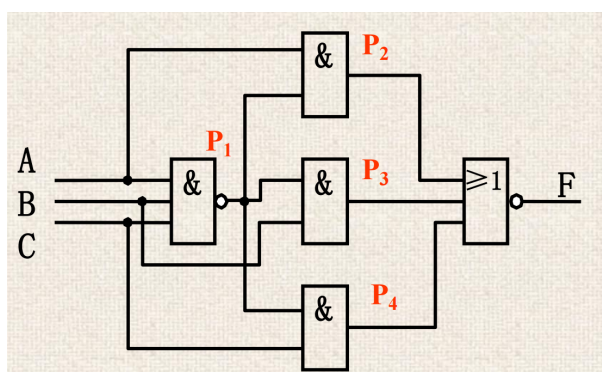
3.1 由基本逻辑门构成的组合电路的设计与分析

3.1.1 一般分析方法

分析步骤：

1. 根据逻辑电路，写出输出逻辑函数的表达式
2. 列出真值表
3. 由真值表或表达式分析电路功能

以下题为例：



采取从后往前分析的方法，把逻辑门输出结果分别标出为 P_1 、 P_2 、 P_3 、 P_4 。从后往前分析每一个逻辑门：

$$F = \overline{P_2 + P_3 + P_4}$$

$$P_2 = AP_1$$

$$\mathbf{P}_3 = \mathbf{B}\mathbf{P}_1$$

$$\mathbf{P}_4 = \mathbf{C}\mathbf{P}_1$$

$$\mathbf{P}_1 = \overline{\mathbf{A}\mathbf{B}\mathbf{C}}$$

$$\mathbf{F} = \overline{(\mathbf{A} + \mathbf{B} + \mathbf{C}) \cdot \overline{\mathbf{A}\mathbf{B}\mathbf{C}}} = \mathbf{A}\mathbf{B}\mathbf{C} + \overline{\mathbf{A}}\overline{\mathbf{B}}\overline{\mathbf{C}}$$

由表达式列出真值表，其中 $\mathbf{P}_1 \sim \mathbf{P}_4$ 为图中各逻辑门输出：

表 3.1: 组合逻辑电路例题真值表

A	B	C	P₁	P₂	P₃	P₄	F = $\overline{\mathbf{P}_2 + \mathbf{P}_3 + \mathbf{P}_4}$
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	1	0	1	0	1	0	0
0	1	1	1	0	1	1	0
1	0	0	1	1	0	0	0
1	0	1	1	1	0	1	0
1	1	0	1	1	1	0	0
1	1	1	0	0	0	0	1

由真值表可知：当且仅当 $\mathbf{A} = \mathbf{B} = \mathbf{C}$ 时 $\mathbf{F} = 1$ ，否则 $\mathbf{F} = 0$ 。即该电路的三个输入取值一致时输出为 1，不一致时输出为 0，实现三变量的“一致”判断（同或）功能。

3.1.2 一般设计方法

一般步骤：

1. 由实际问题列出真值表
2. 由真值表写出逻辑表达式
3. 化简、变换输出逻辑表达式
4. 画出逻辑图

如果现在要设计用与非门的一个三变量表决电路，表决规则为少数服从多数，怎么设计？

设由 **A**、**B**、**C** 表示三个输入变量，**F** 表示表决结果。并设 **A**、**B**、**C** 为 1 表示赞成，为 0 表示反对；**F** 为 1 表示表决通过，为 0 表示不通过。列出真值表：

表 3.2: 三变量表决器真值表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

由表决规则，三个输入中有两个或两个以上为 1 时表决通过，故 **F** = 1 对应 **ABC** 取 011、101、110、111 四种情况。

将 **F** = 1 的四个最小项 m_3 、 m_5 、 m_6 、 m_7 填入卡诺图并圈组，如图所示：

A \ BC	00	01	11	10
0	0	0	1	0
1	0	1	1	1

图 3.1: 三变量表决电路的卡诺图及圈法

图中三个圈分别合并为 **BC**、**AC**、**AB**，故最简与或式为

$$\mathbf{F} = \mathbf{AB} + \mathbf{BC} + \mathbf{AC} = \overline{\overline{\mathbf{A}\mathbf{B}}} \cdot \overline{\overline{\mathbf{B}\mathbf{C}}} \cdot \overline{\overline{\mathbf{A}\mathbf{C}}}$$

下一步画出逻辑图，如下图 3.2所示：

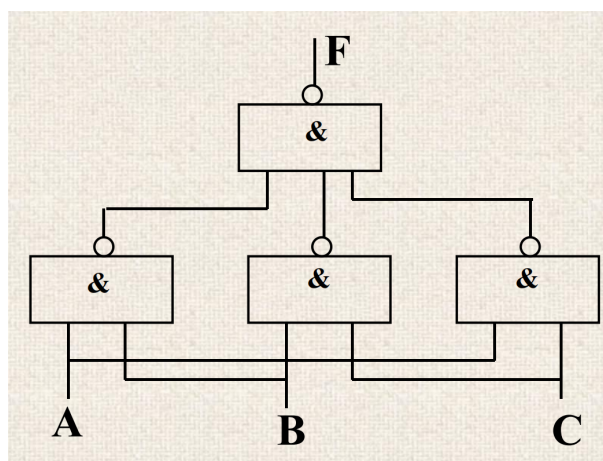


图 3.2: 三变量表决电路的逻辑图

3.2 MSI 构成的组合逻辑电路

3.2.1 自顶向下的模块化设计方法

顶: 指系统功能, 即系统总要求, 较抽象。

向下: 指根据系统总要求, 将系统分解为若干个子系统, 再将每个子系统分解为若干个功能模块, 直至分成许多各具特定功能的基本模块为止。

如果我现在想设计一个数据检测系统, 功能表如下:

表 3.3: 数据检测系统的功能表

S_1	S_2	输出功能
0	0	$A + B$
0	1	$A - B$
1	0	$\text{Min}(A, B)$
1	1	$\text{Max}(A, B)$

其中 S_1 、 S_2 为两个子系统, A 、 B 为传感器数据输入。

3.2.2 编码器

编码器能够将信息（如数和字符等）转换成符合一定规则的二进制代码。

二进制编码器

用 n 位二进制代码对 $N = 2^n$ 个特定信息进行编码的逻辑电路。

下面说明一下二进制编码器的设计方法。

假设咱们现在需要设计一个具有互相排斥输入条件的编码器。

输入为 $\mathbf{X}_0$ 、 $\mathbf{X}_1$ 、 $\mathbf{X}_2$ 、 $\mathbf{X}_3$ ，输出为 $\mathbf{A}_0$ 、 $\mathbf{A}_1$ 。具有如下对应关系：

表 3.4: 二进制编码器的输入输出对应关系

输入	$\mathbf{A}_1$	$\mathbf{A}_0$
$\mathbf{X}_0$	0	0
$\mathbf{X}_1$	0	1
$\mathbf{X}_2$	1	0
$\mathbf{X}_3$	1	1